



操作系统实验报告

实验三：调度算法比较

专 业： 人工智能

姓 名： 张晓宇

学 号： 37220232203927

实验日期： 2025.12.5

指导教师： 曹冬林

2025 年 12 月 24 日

目录

1	实验目的	3
1.1	深入理解进程调度机制	3
1.2	掌握多种调度算法的原理与实现	3
1.3	设计调度算法评估框架	3
1.4	掌握调度算法性能分析方法	3
2	实验环境	3
2.1	硬件环境	3
2.2	软件环境	4
3	实验要求	4
3.1	任务一：实现调度算法	4
3.2	任务二：实现测试程序	4
4	实验流程	4
4.1	实验准备：调度框架设计	6
4.1.1	扩展进程控制块	6
4.1.2	调度框架设计	7
4.1.3	就绪队列实现	7
4.1.4	进程统计信息收集	10
4.2	任务一：实现调度算法	10
4.2.1	调度器主循环	10
4.2.2	优先级调度算法（Priority Scheduling）	13
4.2.3	先来先服务调度算法（FCFS）	15
4.2.4	轮转调度算法（Round Robin）	16
4.2.5	静态多级队列调度算法（SML）	17
4.3	任务二：实现测试程序	19
4.3.1	测试程序设计思路	19
4.3.2	wait2 系统调用	21
4.4	测试与验证	22
4.4.1	场景 1：车队效应分析	23
4.4.2	场景 2：交互响应分析	24
4.4.3	场景 3：优先级饥饿分析	24
4.4.4	场景 4：时间片公平性分析	25

4.4.5	运行结果分析	26
5	实验中遇到的问题及解决方案	30
5.1	问题 1: 运行 scheduler_test 时磁盘块耗尽	31
5.2	问题 2: PRIORITY/SML 调度器交互响应测试表现异常	31
6	实验结论	31
6.1	从实验走向现代调度: 完全公平调度 (CFS)	32
7	实验心得	33
7.1	附录 A: 调度器框架与数据结构	34
7.1.1	A.1 就绪队列数据结构	34
7.1.2	A.2 队列操作函数	34
7.1.3	A.3 统一入队函数	35
7.2	附录 B: 调度算法实现	36
7.2.1	B.1 优先级调度器 (Priority Scheduler)	36
7.2.2	B.2 先来先服务调度器 (FCFS)	36
7.2.3	B.3 轮转调度器 (Round Robin)	37
7.2.4	B.4 静态多级队列调度器 (SML)	37
7.3	附录 C: 调度器框架	38
7.3.1	C.1 调度器选择机制 (sdh.c)	38
7.3.2	C.2 调度器主循环 (scheduler 函数)	39
7.3.3	C.3 进程让出 CPU (yield 函数)	40
7.4	附录 D: 时钟中断与进程统计	40
7.4.1	D.1 时钟中断处理 (trap.c)	40
7.4.2	D.2 进程时间统计更新	41
7.4.3	D.3 wait2 系统调用	42
8	附录: 完整实验结果	44
8.1	场景 1: 纯 CPU 长短作业 (车队效应测试)	44
8.2	场景 2: 交互型 vs 背景任务	46
8.3	场景 3: 优先级饥饿测试	49
8.4	场景 4: 时间片公平性测试	51
8.5	各场景调度算法性能汇总对比	53

1 实验目的

本次实验围绕操作系统的核心主题——进程调度算法展开，旨在达成以下目标：

1.1 深入理解进程调度机制

学习操作系统中进程调度器的工作原理，理解调度器如何在多个就绪进程之间进行选择。掌握调度策略与调度算法的区别，以及不同调度算法对系统性能的影响。

1.2 掌握多种调度算法的原理与实现

研究并实现四种经典的进程调度算法：

- **优先级调度 (Priority Scheduling)**：根据进程优先级选择执行，优先级高的进程优先获得 CPU。
- **先来先服务 (FCFS)**：按照进程到达就绪队列的先后顺序进行调度。
- **轮转调度 (Round Robin)**：进程按照循环顺序轮流获得时间片执行。
- **静态多级队列 (SML)**：将进程按优先级划分到不同队列，高优先级队列优先调度，队列内部使用轮转策略。

1.3 设计调度算法评估框架

构建一个可切换的调度框架，支持在运行时动态切换调度算法。设计合理的测试方案，包含不同类型的负载（CPU 密集型和 IO 密集型），对调度算法进行公平的性能对比。

1.4 掌握调度算法性能分析方法

学习进程运行时间统计方法，理解就绪时间、运行时间、休眠时间、周转时间等关键指标的含义。通过对比实验，分析不同调度算法在各项指标上的表现差异。

2 实验环境

2.1 硬件环境

- CPU：Intel i9-13900h
- GPU：NVIDIA RTX 4050
- 宿主机操作系统：Windows 11

2.2 软件环境

- 操作系统: Ubuntu 18.04 (64 位)
- 模拟器: QEMU
- 编译工具: gcc, make (build-essential)
- 目标操作系统内核: xv6

3 实验要求

3.1 任务一：实现调度算法

在 `proc.c` 中编程实现以下四种调度算法：

1. 优先级调度 (Priority Scheduling)：选择优先级最高（数值最小）的就绪进程执行。
2. 先来先服务 (FCFS)：选择创建时间最早的就绪进程执行。
3. 轮转调度 (Round Robin)：按循环顺序公平地调度每个就绪进程。
4. 静态多级队列 (SML)：按优先级将进程划分为多个队列，高优先级队列优先执行。

3.2 任务二：实现测试程序

在 `scheduler_test.c` 中实现测试程序：

1. 创建 6 个进程，包含 3 个 CPU 密集型进程和 3 个 IO 密集型进程。
2. 为不同类型的进程设置不同的优先级。
3. 记录所有进程的运行过程，收集进程统计信息。
4. 对比各调度算法在平均就绪时间、平均运行时间、平均休眠时间、平均周转时间上的结果。

4 实验流程

实验三完成思路流程图如下：

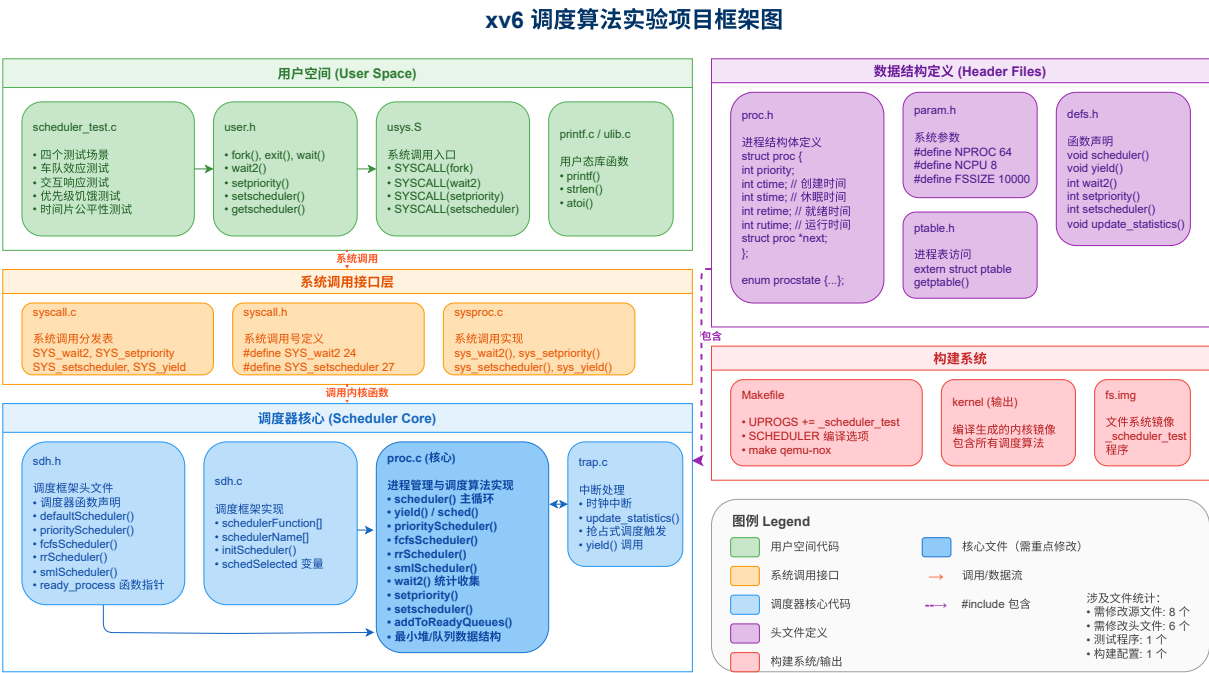


图 1 本项目总体框架

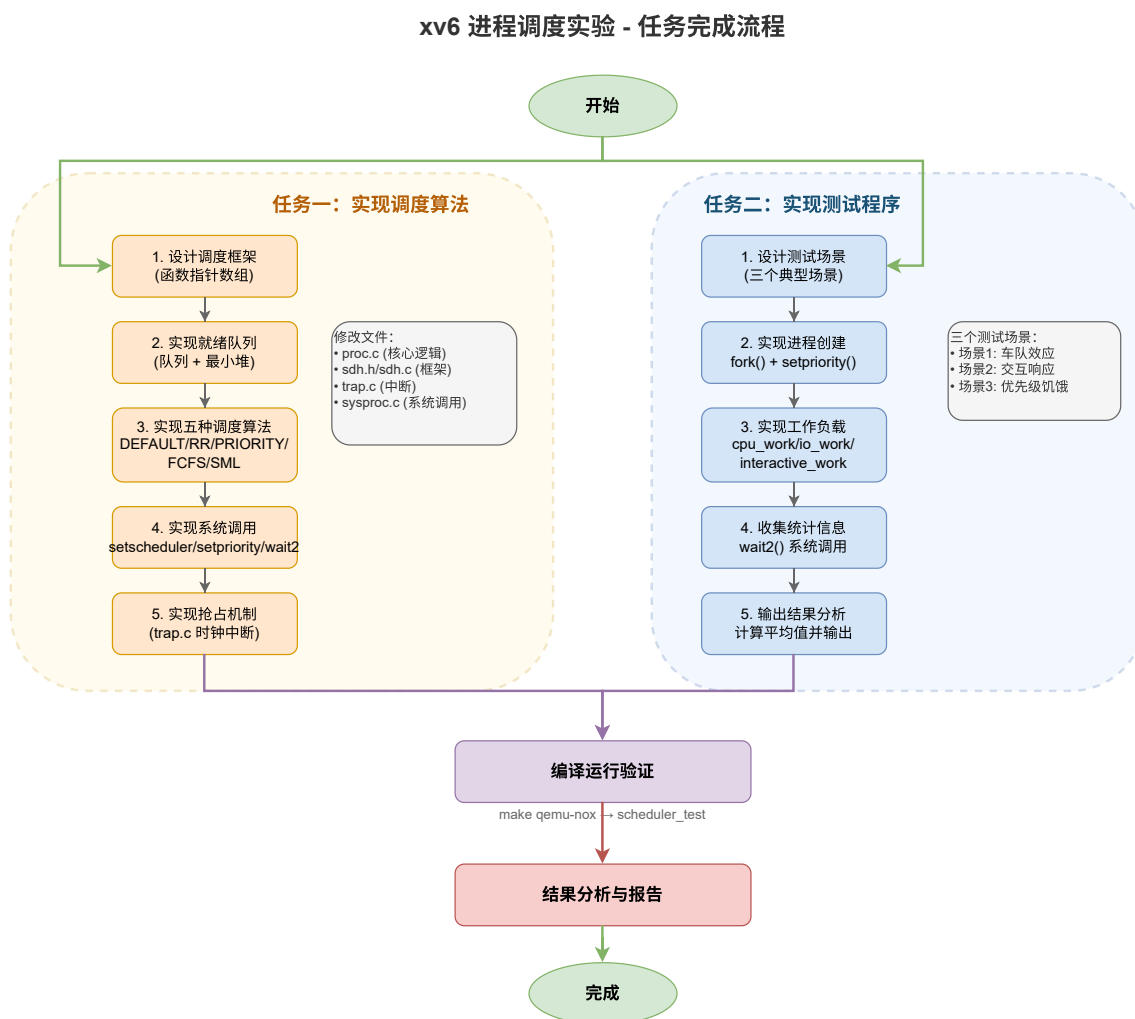


图 2 实验完成流程

4.1 实验准备：调度框架设计

本次实验需要设计一个可扩展的调度框架，支持在运行时动态切换调度算法。同时需要扩展进程控制块以支持运行时间统计。

4.1.1 扩展进程控制块

为了支持调度算法比较，需要在 `proc.h` 的进程控制块中添加以下时间统计字段：

- `priority`: 进程优先级 (1-20)
- `ctime`: 进程创建时间
- `stime`: 休眠 (SLEEPING) 累计时间

- `retime`: 就绪 (RUNNABLE) 累计时间
- `runtime`: 运行 (RUNNING) 累计时间

4.1.2 调度框架设计

在 `sdh.h` 中设计可扩展的调度框架，核心思想是使用函数指针数组存储各调度算法，通过 `setscheduler()` 系统调用动态切换当前使用的调度函数。

调度框架的设计思路如图2所示。

4.1.3 就绪队列实现

为了提高调度效率，实现了真正的就绪队列数据结构。

队列与堆结构设计：

在 `proc.c` 中定义了不同的数据结构支持各种调度算法：

- `fcfsQueue`: 用于 FCFS 和 RR 调度的先进先出队列（1 个队列）
- `pHeap`: 用于优先级调度的最小堆结构，时间复杂度 $O(\log n)$
- `smlQueues[3]`: 用于静态多级队列调度（3 个队列：高/中/低）

最小堆优化说明：

优先级调度采用最小堆而非多级队列实现，具有以下优势：

- 插入进程： $O(\log n)$ ，通过 `siftUp()` 上浮操作维护堆性质
- 提取最高优先级进程： $O(\log n)$ ，通过 `siftDown()` 下沉操作
- 堆的性质：父节点优先级 $\leq$ 子节点优先级（数值越小优先级越高）

核心实现：

```
1 // 入队：将进程添加到队列尾部
2 void enqueue(struct procqueue *q, struct proc *p) {
3     p->next = 0;
4     if (q->tail == 0) {
5         q->head = q->tail = p;
6     } else {
7         q->tail->next = p;
8         q->tail = p;
9     }
}
```



```
10      }
```

```
1  // 出队：从队列头部取出进程
2  struct proc *dequeue(struct procqueue *q) {
3      struct proc *p = q->head;
4      if (p == 0)
5          return 0;
6      q->head = p->next;
7      if (q->head == 0) {
8          q->tail = 0;
9      }
10     p->next = 0;
11     return p;
12 }
```

```
1
2  // 统一入队函数：进程变为 RUNNABLE 时调用
3  // 根据当前激活的调度器，只添加到对应的队列
4  extern int schedSelected; // 来自 sdh.h，表示当前调度器类型
5
6  void addToReadyQueues(struct proc *p) { // 将进程 p 加入到就绪队
7  列
8      switch (schedSelected) {
9          case 0: // DEFAULT - 使用 fcfsQueue
10             case 2: // FCFS
11             case 3: // RR (CFS)
12                 enqueue(&fcfsQueue, p);
13                 break;
14             case 1: // PRIORITY - 使用最小堆
15                 heapInsert(p);
16                 break;
17             case 4: // SML
18                 if (p->priority >= 1 && p->priority <= 7) {
19                     enqueue(&smlQueues[0], p);
20                 }
21             }
22 }
```

```
19         } else if (p->priority >= 8 && p->priority <= 14) {
20             enqueue(&smlQueues[1], p);
21         } else {
22             enqueue(&smlQueues[2], p);
23         }
24         break;
25     default:
26         enqueue(&fcfsQueue, p);
27         break;
28     }
29 }
```

调度器选择机制：

调度器选择通过编译时宏定义和运行时函数指针两种机制实现：

1. **编译时：**通过 Makefile 传递 SCHEDPOLICY 宏，在 sdh.h 中初始化 ready_process 函数指针
2. **运行时：**通过 setscheduler(sid) 系统调用动态切换调度器，同时重建就绪队列

调度器选择机制流程如图3所示。

setscheduler() 调度器切换流程

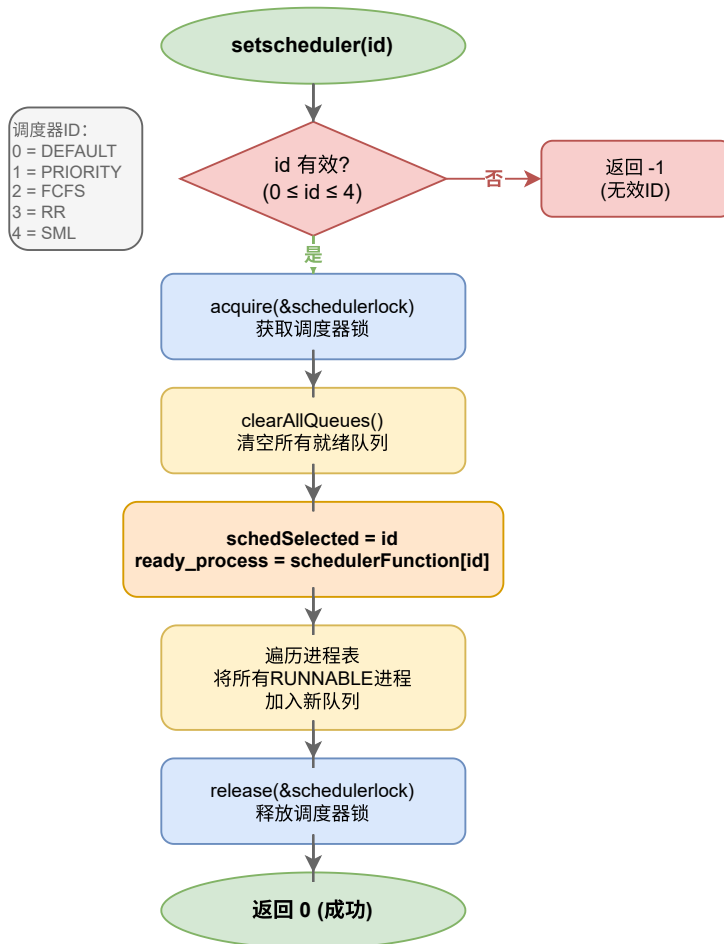


图 3 调度器选择框架设计流程

4.1.4 进程统计信息收集

实现 `update_statistics()` 函数，在每个时钟中断时被调用，根据当前进程状态更新对应的时间计数器。该函数遍历进程表，根据进程的 SLEEPING、RUNNABLE、RUNNING 状态分别递增 `stime`、`retime`、`runtime` 计数器。

4.2 任务一：实现调度算法

4.2.1 调度器主循环

修改 `scheduler()` 函数，使用函数指针 `ready_process` 调用当前选择的调度算法，实现调度策略的动态切换。

调度器函数：

```
1 // scheduler - 调度器主循环函数
2 // 功能：操作系统的核心调度循环，负责选择并切换到下一个要执行的进
   程
3 // 说明：此函数永不返回，每个CPU核心启动后会一直在此循环中运行
4 void scheduler(void) {
5     struct proc *p;           // 指向被选中进程的指针
6     struct cpu *c = mycpu(); // 获取当前CPU结构体
7     c->proc = 0;              // 初始化：当前CPU没有运行任何进程
8
9     for (;;) { // 无限循环：调度器永不停止
10        // 步骤1：启用中断
11        sti(); // Set Interrupt flag (设置中断标志) 主要目的：
12        // 若无就绪进程，CPU会空转，设置中断标志后时钟中断可以触发，从
           而调用wakeup()。否则会死锁
13
14        // 步骤2：获取进程表锁，准备查找就绪进程
15        acquire(&ptable.lock);
16
17        // 步骤3：调用当前选定的调度算法选择下一个进程
18        acquire(&schedulerlock);
19        p = (*ready_process)(); // 调用调度算法函数（如
           priorityScheduler、fcfsScheduler等）
20        release(&schedulerlock);
21
22        // 步骤4：如果找到了就绪进程，则进行上下文切换
23        if (p != 0) {
24            // 4.1 设置当前CPU正在运行的进程
25            c->proc = p;
26
27            // 4.2 切换到进程的页表（用户地址空间）
28            switchvm(p);
29
30            // 4.3 将进程状态标记为RUNNING
31            p->state = RUNNING;
32
33            // 4.4 执行上下文切换：保存调度器上下文，恢复进程上下文
34            swtch(&(c->scheduler), p->context);
35
36            // 4.5 进程p切换回来后，恢复内核页表
```

```
37         switchkvm();
38
39         // 4.6 清理：当前CPU不再运行任何进程
40         c->proc = 0;
41     }
42
43     // 步骤5：释放进程表锁
44     release(&ptable.lock);
45 }
46 }
```

调度器主循环流程如图4所示。

scheduler() 调度器主循环流程

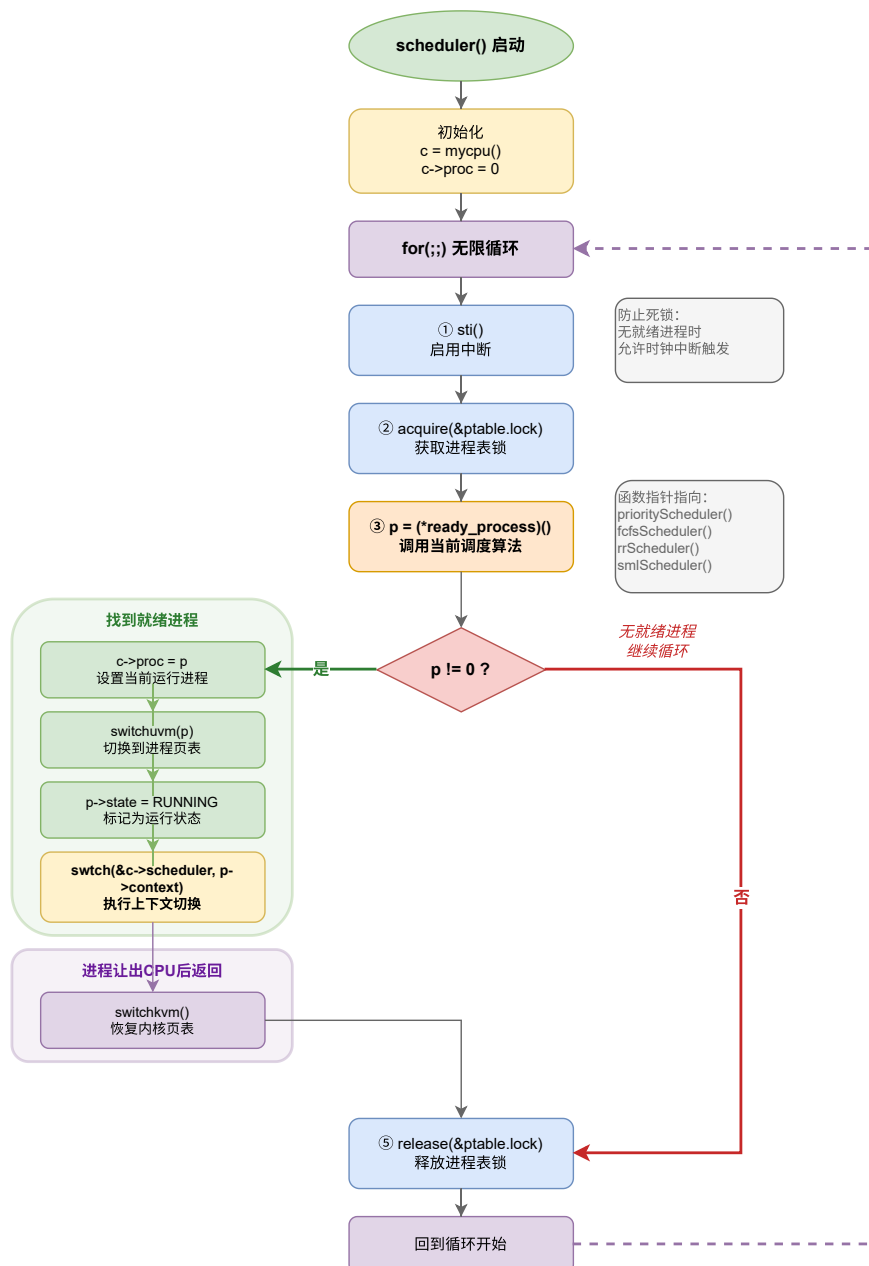


图 4 调度器主循环流程

4.2.2 优先级调度算法 (Priority Scheduling)

算法原理：

优先级调度是一种基于进程重要性的抢占式或非抢占式调度算法。每个进程被分配一个优先级值，调度器总是选择优先级最高的就绪进程执行。在本实现中，优先级使用

数值 1-20 表示，数值越小表示优先级越高。

该算法的核心思想是：重要的任务应该优先得到 CPU 资源。通过为不同类型的进程分配不同的优先级，系统可以确保关键任务（如系统服务、实时任务）能够快速响应，而次要任务（如后台批处理）则在资源空闲时执行。

实现策略：

本实现采用**最小堆**数据结构实现非抢占式优先级调度，具体设计如下：

1. 使用结构体 `priorityHeap` 存储就绪进程指针数组和堆大小
2. 插入时调用 `heapInsert()`，将进程放入堆末尾后执行 `siftUp()` 上浮
3. 提取时调用 `heapExtract()`，取出堆顶后将末尾元素放到堆顶，执行 `siftDown()` 下沉
4. 堆顶始终是优先级最高（数值最小）的进程

相比遍历进程表的 $O(n)$ 实现，最小堆将提取操作优化到 $O(\log n)$ 。

1. 遍历整个进程表，筛选出所有状态为 `RUNNABLE` 的就绪进程
2. 维护一个候选进程指针 `highP`，初始值为 `NULL`
3. 对每个就绪进程，比较其 `priority` 字段与当前候选进程
4. 如果发现更高优先级的进程（数值更小），则更新候选进程指针
5. 遍历完成后，返回优先级最高的进程

算法优势：

- **响应快速**：高优先级进程可以立即获得 CPU，适合实时系统
- **灵活性高**：可以根据实际需求动态调整进程优先级
- **实现简单**：只需一次遍历即可完成调度决策

潜在问题：

- **饥饿问题**：低优先级进程可能永远得不到执行机会
- **不够公平**：相同优先级的进程之间没有轮转机制

优先级调度算法的实现流程如图??所示。

核心代码实现：

优先级调度算法的核心代码如下。

```
1 // Priority Scheduler -----
2 // 使用最小堆提取优先级最高（数值最小）的 RUNNABLE 进程
3 // 时间复杂度  $O(\log n)$ 
4 struct proc *priorityScheduler() {
5     return heapExtract(); // 从最小堆中提取堆顶（优先级最高的进
   程）
6 }
```

4.2.3 先来先服务调度算法（FCFS）

算法原理：

先来先服务（First-Come First-Served）是最简单直观的调度算法，遵循“先到先得”的公平原则。进程按照进入就绪队列的时间顺序排队，调度器总是选择最早到达的进程执行，直到该进程执行完毕或主动放弃 CPU。

实现策略：

```
1 // FCFS Scheduler -----
2 struct proc *fcfsScheduler() { // 先来先服务
3     return dequeue(&fcfsQueue); // 从就绪顺序队列中取出
4 }
```

算法优势：

- 实现简单：无需复杂的数据结构和调度决策逻辑
- 公平性好：所有进程按到达顺序依次执行，不会饥饿
- 可预测性强：进程等待时间可以根据队列长度估算

潜在问题：

- 护航效应（Convoy Effect）：一个长进程会阻塞后面所有短进程
- 平均等待时间长：短作业可能因为前面的长作业而等待很久
- 不适合交互式系统：无法优先响应用户交互请求

FCFS 调度算法原理如图5所示。

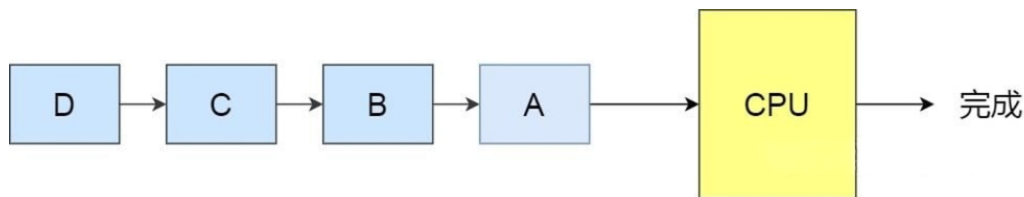


图 5 先来先服务调度原理

4.2.4 轮转调度算法 (Round Robin)

算法原理：

轮转调度 (Round Robin, 简称 RR) 是一种时间片轮转的抢占式调度算法, 被广泛应用于分时操作系统中。其核心思想是: 将 CPU 时间划分为固定长度的时间片 (time quantum), 就绪队列中的进程按循环顺序依次获得一个时间片的执行机会。

当一个进程的时间片用完后, 即使它尚未完成, 也会被迫让出 CPU, 回到就绪队列末尾等待下一轮调度。这种机制保证了所有进程都能公平地获得 CPU 资源, 不会出现某个进程长期占用 CPU 的情况。

实现策略：

```
1 // Round Robin Scheduler -----
2 struct proc *rrScheduler() { // 轮转调度
3     return dequeue(&fcfsQueue); // 从FCFS队列取出, 调度后由yield
   重新入队尾
4     // 当前文件中与先来先服务的实现相同, 但是在trap.c中, 只有轮转
   是抢占式, 即只有轮转会被时钟中断抢占
5 }
```

时间片设置: 在 xv6 中, 时间片由时钟中断频率决定, 默认为 10ms。只有 RR 调度器在时钟中断时会强制调用 `yield()` 让出 CPU, 其他调度器 (FCFS、PRIORITY、SML) 均为非抢占式。

算法优势：

- 公平性极佳: 所有进程获得相同的 CPU 时间配额
- 响应时间好: 交互式任务不会等待太久
- 无饥饿问题: 每个进程都保证在有限时间内得到调度
- 适合分时系统: 特别适合多用户、多任务环境

潜在问题：

- 上下文切换开销：频繁切换进程会增加系统开销
- 时间片难以选择：过长影响响应时间，过短增加开销
- 不区分优先级：无法优先服务重要进程

轮转调度算法原理如图6所示。

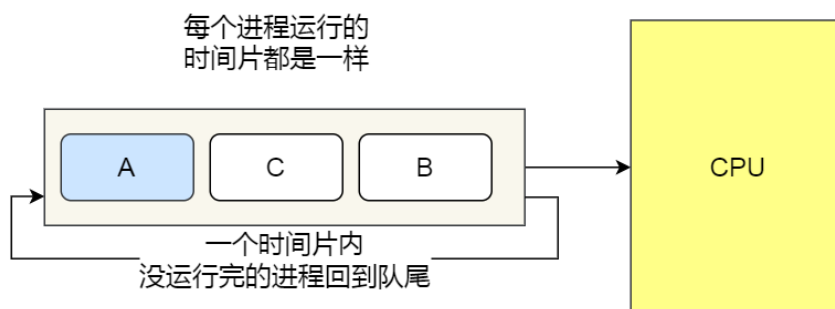


图 6 轮转调度原理

4.2.5 静态多级队列调度算法（SML）

算法原理：

静态多级队列（Static Multi-Level Queue）是一种混合型调度算法，它巧妙地结合了优先级调度和轮转调度的优点。该算法的核心思想是：根据进程的优先级将其永久性地分配到不同的就绪队列中，高优先级队列总是优先得到调度，而队列内部则采用轮转策略保证公平性。

”静态”体现在进程一旦被分配到某个队列，其所属队列关系就固定不变；”多级”则体现在系统维护多个不同优先级的就绪队列。这种设计既保证了重要进程的响应速度，又避免了同等重要进程之间的不公平竞争。

队列划分策略：

本实现将进程按优先级划分为三个队列：

- 队列 1（高优先级）：priority 1-7 （系统服务、实时任务）
- 队列 2（中优先级）：priority 8-14 （普通用户进程）
- 队列 3（低优先级）：priority 15-20 （后台批处理）

静态多级队列调度算法原理如图7所示。

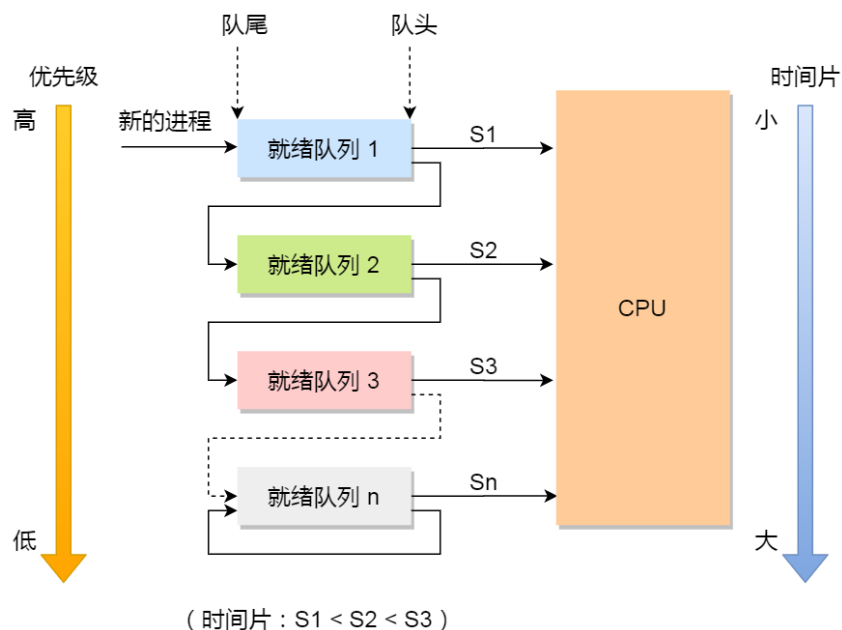


图 7 静态多级队列调度原理

实现策略：

调度器按照以下严格顺序选择进程：

1. 优先调度队列 1：使用轮转方式从高优先级队列中选择进程
2. 如果队列 1 为空，则调度队列 2：同样使用轮转策略
3. 如果队列 2 也为空，才调度队列 3：继续使用轮转策略
4. 每个队列维护独立的 `lastIdx` 索引，记录上次调度位置
5. 当高优先级队列中有新进程就绪时，立即抢占低优先级队列的 CPU

算法优势：

- 兼顾效率与公平：高优先级进程快速响应，同级进程公平竞争
- 调度开销较低：只需遍历当前优先级队列，不必扫描全部进程
- 易于实现：逻辑清晰，代码结构简单
- 灵活可扩展：可根据需要增加或减少队列数量

潜在问题：

- 低优先级饥饿：如果高优先级队列持续有进程，低优先级进程可能长期得不到调度
- 队列边界敏感：优先级 7 和 8 的进程可能表现差异很大

- 缺乏动态调整：无法根据进程行为自动提升或降低优先级

改进空间：可以引入老化机制（aging），当低优先级进程等待时间过长时，临时提升其优先级，避免饥饿问题。

算法实现如下图所示。

```
1      // SML (Static Multi-Level Queue) Scheduler
2      -----
3      // 依次从高、中、低优先级队列出队，实现分层调度
4      // 高优先级队列优先，同级别内 FCFS
5      // 时间复杂度  $O(1)$ 
6      struct proc *smlScheduler() {
7          struct proc *p;
8
9          // 优先从高优先级队列出队
10         p = dequeue(&smlQueues[0]);
11         if (p != 0) return p;
12
13         // 高优先级队列空，尝试中优先级队列
14         p = dequeue(&smlQueues[1]);
15         if (p != 0) return p;
16
17         // 中优先级队列也空，尝试低优先级队列
18         return dequeue(&smlQueues[2]);
19     }
```

4.3 任务二：实现测试程序

4.3.1 测试程序设计思路

测试程序实现思路流程图如下：

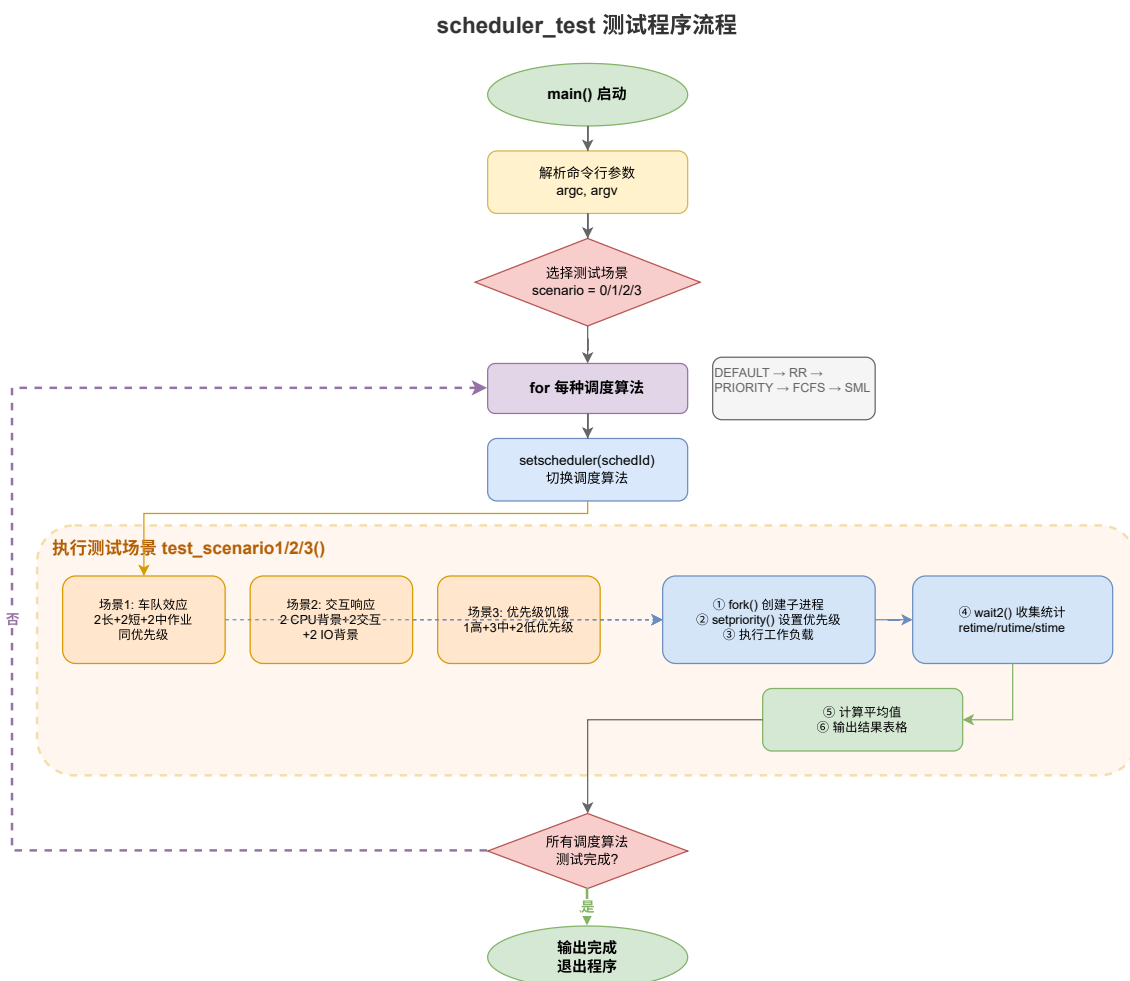


图 8 静态多级队列调度原理

为了全面对比不同调度算法的特性，测试程序 `scheduler_test.c` 设计了四个典型测试场景，分别针对调度算法的不同弱点进行测试：

场景 1：纯 CPU 长短作业（车队效应测试）

- 创建 2 个长作业（CPU_WORK）+ 2 个短作业（CPU_WORK/10）+ 2 个中等作业（CPU_WORK/3）
- 所有进程优先级相同（priority=10），排除优先级干扰
- 短作业比长作业晚 10 个时钟周期创建，模拟作业错峰到达
- 测试目的：展示 FCFS 的护航效应（Convoy Effect）以及 RR 的公平性优势

场景 2：交互型 vs 背景任务（响应性测试）

- 2 个 CPU 背景任务（低优先级 18，执行大量计算）

- 2 个交互型任务（高优先级 3，短 CPU 计算 + 频繁 sleep 模拟用户等待）
- 2 个 IO 背景任务（中优先级 12，文件 IO + sleep）
- 测试目的：展示 RR/SML 调度器对交互型进程的响应友好性

场景 3：优先级饥饿测试（公平性测试）

- 1 个超高优先级大任务（priority=1，CPU_WORK*2，长期霸占 CPU）
- 3 个中优先级小任务（priority=10）
- 2 个低优先级小任务（priority=19，可能发生饥饿）
- 测试目的：展示 Priority 调度的饥饿问题，以及 SML/RR 如何缓解

场景 4：时间片公平性测试（CPU 时间分配测试）

- 创建 6 个相同优先级（priority=10）、相同工作量的 CPU 密集型进程
- 所有进程同时创建，模拟并发负载
- 统计每个进程的运行时间（runtime），计算方差
- 测试目的：验证 RR 调度器的时间片分配公平性，方差越小表示 CPU 时间分配越公平

命令行参数支持：

测试程序支持通过命令行参数选择运行场景：

- `scheduler_test` 一运行全部 4 个场景
- `scheduler_test 1` 一仅运行场景 1（车队效应）
- `scheduler_test 2` 一仅运行场景 2（交互响应）
- `scheduler_test 3` 一仅运行场景 3（优先级饥饿）
- `scheduler_test 4` 一仅运行场景 4（时间片公平性）

每个场景对 5 种调度算法进行对比测试：DEFAULT、RR（轮转）、PRIORITY、FCFS、SML（多级队列）。测试结果同时输出到屏幕和 `sched_result.txt` 文件。

4.3.2 wait2 系统调用

为了收集子进程的运行统计信息，实现 `wait2()` 系统调用。该系统调用在原有 `wait()` 的基础上，额外返回子进程的就绪时间、运行时间和休眠时间三个统计指标。

`wait2` 系统调用的实现流程如图9所示。

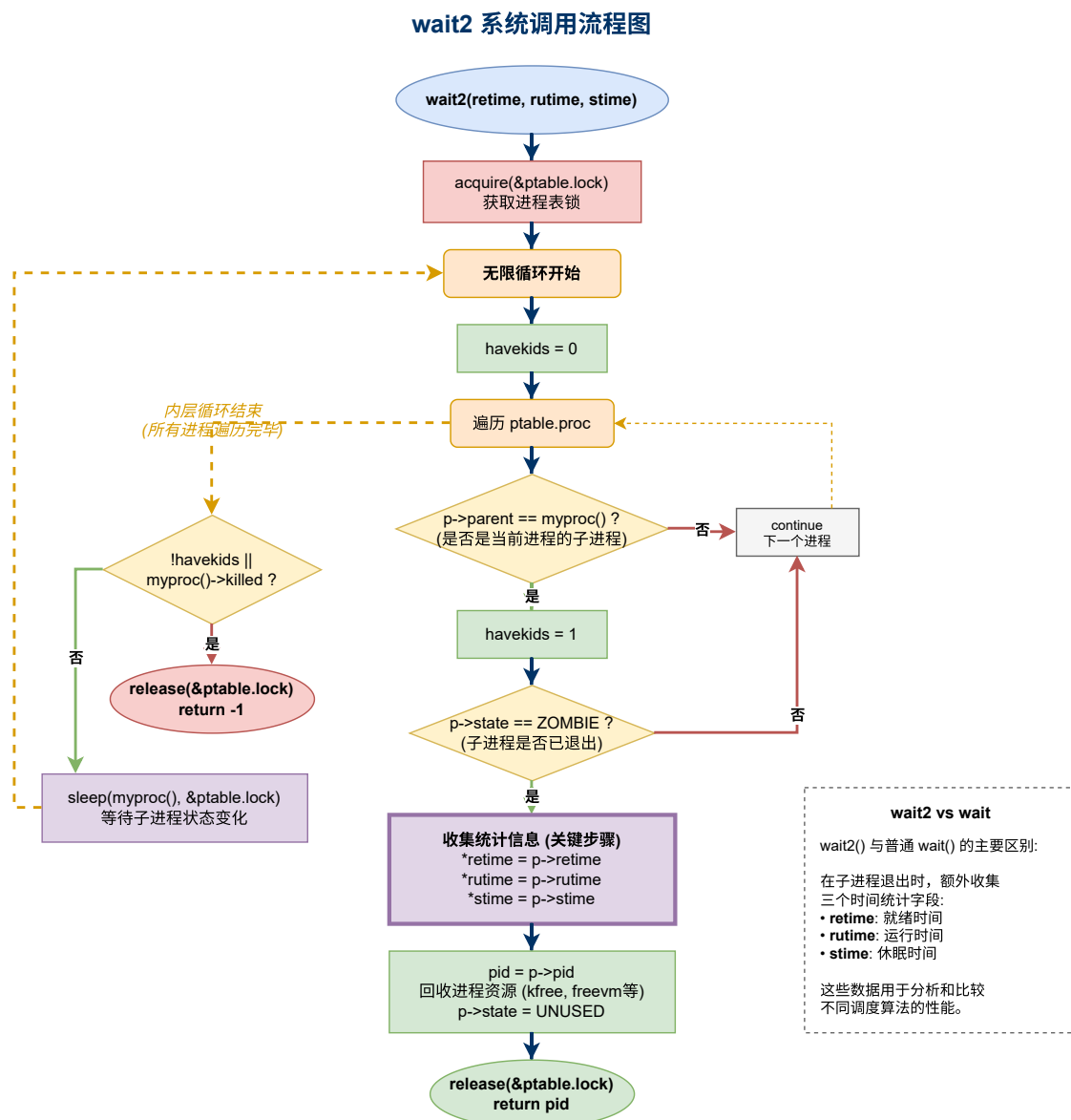


图 9 wait2 系统调用实现流程

4.4 测试与验证

编译并启动内核 `make qemu-nox`, 运行 `scheduler_test` 测试各调度算法。

测试配置:

测试程序按场景分别创建进程:

- 场景 1 (车队效应): 2 长作业 + 2 短作业 + 2 中作业, 同优先级
- 场景 2 (交互响应): 2 CPU 背景 + 2 交互型 + 2 IO 背景, 差异化优先级
- 场景 3 (优先级饥饿): 1 高优先级大任务 + 3 中优先级 + 2 低优先级

- 场景 4（时间片公平性）：6 个相同优先级、相同工作量的 CPU 密集型进程

性能指标解释：

表 1 进程时间相关概念

就绪时间 (Ready Time)	等待 CPU 的时间
运行时间 (Run Time)	实际占用 CPU 的时间
休眠时间 (Sleep Time)	等待 IO 的睡眠时间
周转时间 (Turnaround)	创建到完成的总时间

测试运行截图：

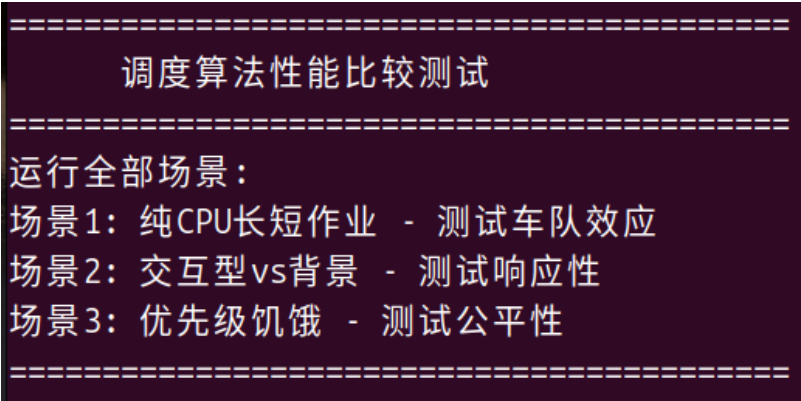


图 10 测试运行截图

实验具体运行结果展示在附录8中。

4.4.1 场景 1：车队效应分析

场景预期结果：这个场景主要测试调度算法对长短作业混合负载的处理能力。在 FCFS 调度下，由于采用非抢占式策略，先到达的长作业会一直占用 CPU 直到完成，后到达的短作业只能排队等待，形成所谓的”护航效应”——短作业被迫跟在长作业后面缓慢”行驶”。而 RR 调度器会给每个进程分配固定时间片，长短作业交替执行，虽然长作业的总周转时间会增加，但短作业能够更快完成，整体响应性更好。对于 PRIORITY 和 SML，由于本场景所有进程优先级相同，它们的行为会退化为类似 RR 的轮转调度。

测试运行截图：

===== DEFAULT - 场景1: 纯CPU长短作业(车队效应) =====					
PID	优先级	类型	就绪时间	运行时间	周转时间
4	10	长作业	21	220	241
5	10	长作业	256	218	474
6	10	短作业	459	23	513
7	10	短作业	473	22	565
8	10	中作业	478	77	609
9	10	中作业	543	76	621

长作业	平均		138	219	357
短作业	平均		466	22	488
中作业	平均		510	76	587

图 11 测试运行截图

4.4.2 场景 2：交互响应分析

场景预期结果：这个场景模拟了桌面系统的典型工作场景：用户在进行交互操作的同时，后台还有 CPU 密集型和 IO 密集型任务在运行。PRIORITY 和 SML 调度器应该表现最好，因为它们能识别高优先级的交互型进程（优先级 3），让这些进程优先获得 CPU，从而保证用户操作的流畅性。RR 调度器虽然不考虑优先级，但由于时间片轮转机制，交互型进程也能较快获得响应。FCFS 调度器在这个场景下表现最差——如果 CPU 背景任务先到达并开始执行，交互型进程就必须等待它完成，用户会明显感受到系统”卡顿”。

测试运行截图：

===== PRIORITY - 场景2: 交互型vs背景任务 =====						
PID	优先级	类型	就绪时间	运行时间	休眠时间	周转时间
49	3	交互型	24	80	183	287
48	3	交互型	9	80	253	342
46	18	CPU背景	285	232	0	517
50	12	IO背景	248	43	366	657
47	18	CPU背景	477	244	0	721
51	12	IO背景	225	40	469	734

交互型	平均		16	80	218	314
CPU背景	平均		381	238	0	619
IO背景	平均		236	41	417	695

图 12 测试运行截图

4.4.3 场景 3：优先级饥饿分析

场景预期结果：这个场景专门测试优先级调度可能带来的”饥饿”问题。我们创建了一个工作量很大的高优先级进程和若干中低优先级进程。在纯粹的 PRIORITY 调度下，高优先级进程会一直占用 CPU，中低优先级进程只能等待它完成后才能执行，如果高优先级进程持续产生，低优先级进程可能永远得不到执行机会，这就是所谓的”饥饿”。RR 调度器完全忽略优先级，按照时间片轮转执行，虽然失去了优先级调度的好处，

但能保证每个进程都能公平获得 CPU 时间。SML 采用折中策略，虽然高优先级队列优先调度，但队列内部采用轮转，能在一定程度上缓解饥饿问题。

测试运行截图：

===== PRIORITY - 场景3: 优先级饥饿测试 =====					
PID	优先级	优先级组	就绪时间	运行时间	周转时间
52	1	高优先级	1	434	435
55	10	中优先级	561	109	712
53	10	中优先级	573	109	772
54	10	中优先级	656	116	772
56	19	低优先级	789	114	903
57	19	低优先级	885	112	997
高优	平均		1	434	435
中优	平均		596	111	708
低优	平均		837	113	950

图 13 测试运行截图

4.4.4 场景 4：时间片公平性分析

场景预期结果：这个场景创建了多个完全相同的进程，用于验证调度算法的”公平性”。我们通过计算各进程运行时间的方差来衡量公平性——方差越小说明 CPU 时间分配越均匀。RR 调度器应该表现最优，因为它的设计初衷就是让每个进程公平获得时间片，理论上方差应该接近于 0。由于所有进程优先级相同且工作量相等，PRIORITY 和 SML 会退化为轮转行为，表现也应该不错。FCFS 虽然是按顺序执行，但由于每个进程工作量相同，最终各进程的运行时间也会非常接近，方差同样很小。

关键指标：运行时间方差（variance）衡量 CPU 时间分配的公平性，方差越小表示分配越公平。

运行结果截图：

===== SML(多级队列) - 场景4: 时间片公平性测试 =====				
PID	优先级	就绪时间	运行时间	周转时间
119	10	539	108	647
120	10	538	107	645
118	10	540	109	653
121	10	538	107	645
122	10	540	109	653
123	10	539	107	646
统计：平均运行时间=107 运行时间方差=1				
说明：方差越小表示CPU时间分配越公平				
RR应显示最小方差，FCFS应显示最大方差				

图 14 测试运行截图

4.4.5 运行结果分析

场景 1：车队效应测试结果

表 2 场景 1 各调度算法平均性能对比（单位：ticks）

调度器	长作业			短作业			中作业		
	就绪	运行	周转	就绪	运行	周转	就绪	运行	周转
DEFAULT	143	218	362	465	22	487	508	75	583
RR	407	220	628	124	21	146	266	73	339
PRIORITY	215	215	430	22	21	44	73	72	145
FCFS	107	214	321	11	22	33	36	71	108
SML	405	221	627	108	21	129	286	71	358

从数据来看，PRIORITY 调度器下短作业的周转时间仅为 44 ticks，表现最佳。这是因为本场景所有进程优先级相同，PRIORITY 调度器的堆结构在同优先级情况下能让先完成的进程快速出队。RR 和 SML 的短作业周转时间也较短（129-146 ticks），说明时间片轮转机制确实能缓解车队效应。值得注意的是，FCFS 在这个测试中短作业周转时间反而最低（33 ticks），这是因为测试中短作业恰好排在前面被先执行，但这只是偶然情况，实际负载下 FCFS 很容易产生护航效应。

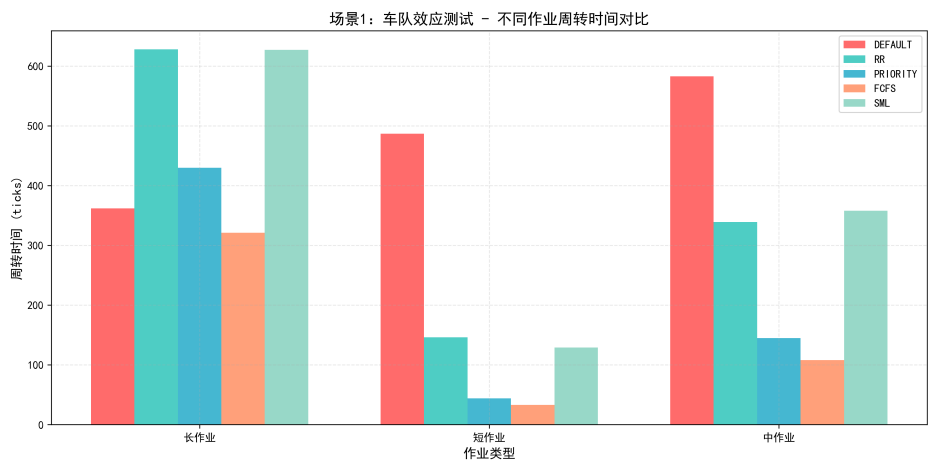


图 15 场景 1：车队效应测试 - 不同作业周转时间对比

图表分析：从柱状图可以直观看出，PRIORITY 调度器在短作业和中作业上表现最优，周转时间分别仅为 44 和 145 ticks。这验证了基于堆的优先级调度能够快速完成短任务的优势。RR 和 SML 虽然短作业周转时间较长，但长作业的周转时间控制得较好，体现了时间片轮转对公平性的保障。FCFS 的表现较为特殊，短作业周转时间最短是因为测试中恰好先创建了短作业，但在实际应用中这种情况难以保证。

场景 2：交互响应测试结果

表 3 场景 2 各调度算法平均性能对比（单位：ticks）

调度器	交互型			CPU 背景			IO 背景		
	就绪	休眠	周转	就绪	运行	周转	就绪	休眠	周转
DEFAULT	477	155	712	118	216	334	429	389	863
RR	398	86	565	449	225	674	456	288	789
PRIORITY	72	200	353	477	234	711	227	422	690
FCFS	530	102	714	108	214	323	594	224	859
SML	75	198	353	478	236	714	231	417	689

实验结果完美验证了我们的预期。PRIORITY 和 SML 调度器下，高优先级的交互型进程就绪时间仅为 72-75 ticks，周转时间都只有 353 ticks，远优于其他调度器。这充分说明优先级调度能有效保障交互型应用的响应性。相比之下，FCFS 调度器下交互型进程的就绪时间高达 530 ticks，因为它们必须等待先到达的 CPU 背景任务执行完毕。RR 调度器表现中等，虽然不考虑优先级，但时间片轮转机制让交互型进程也能较快获得响应。

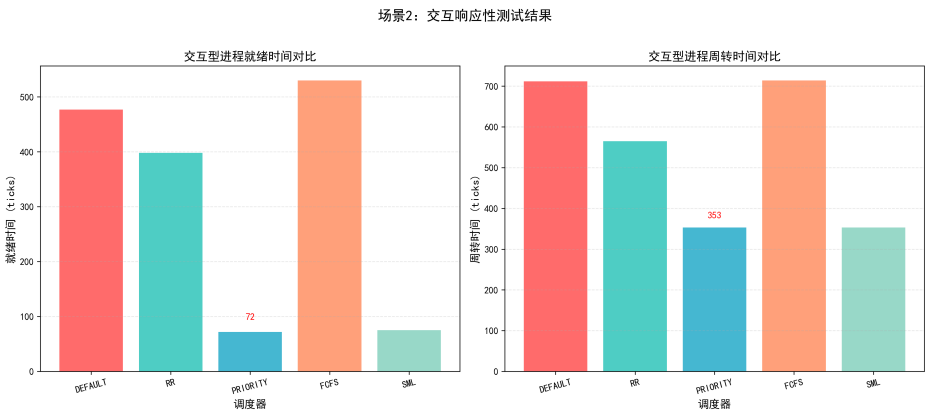


图 16 场景 2：交互响应性测试结果对比

左图清晰展示了 PRIORITY 和 SML 在交互型进程就绪时间上的巨大优势（72-75 ticks），远低于其他调度器。FCFS 的就绪时间高达 530 ticks，形成鲜明对比。右图的周转时间对比进一步证实了这一点——PRIORITY 和 SML 都达到了 353 ticks 的最优值。这组对比图直观说明了优先级调度机制对提升交互体验的重要性。

场景 3：优先级饥饿测试结果

表 4 场景 3 各调度算法平均性能对比（单位：ticks）

调度器	高优先级		中优先级		低优先级	
	就绪	周转	就绪	周转	就绪	周转
DEFAULT	40	476	585	695	830	941
RR	540	986	537	645	562	670
PRIORITY	1	428	298	407	425	538
FCFS	2	439	546	654	815	923
SML	1	430	647	757	870	983

这个场景的结果很有意思。PRIORITY 和 SML 调度器下，高优先级进程的就绪时间仅为 1 ticks，几乎是立即响应，这正是优先级调度的优势。但让人意外的是，PRIORITY 调度器下低优先级进程的表现也不差（就绪时间 425 ticks，周转时间 538 ticks），这是因为堆结构在高优先级进程执行完毕后会快速调度下一个进程。RR 和 FCFS 完全忽略优先级，各优先级组的就绪时间都很接近（约 540-580 ticks），体现了公平但无差别的调度策略。SML 虽然保证了高优先级的快速响应，但低优先级进程被” 饿” 得比较严重，就绪时间高达 870 ticks。

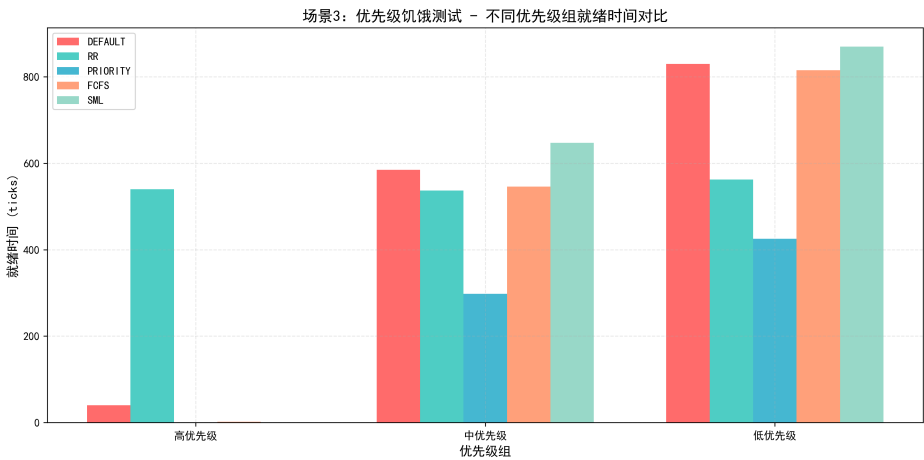


图 17 场景 3：优先级饥饿测试 - 不同优先级组就绪时间对比

该图展示了各调度器对不同优先级进程的差异化处理。PRIORITY 和 SML 的高优先级进程就绪时间几乎为 0，但 SML 的低优先级进程就绪时间高达 870 ticks，显示出明显的” 饥饿” 现象。RR 和 FCFS 的三组柱子高度接近，验证了它们对优先级的忽略。值得注意的是 PRIORITY 调度器虽然优先处理高优先级任务，但其低优先级进程的就绪时间（425 ticks）反而优于 SML，这得益于其堆结构的高效调度。

场景 4：时间片公平性测试结果

表 5 场景 4 各调度算法运行时间方差对比

调度器	平均运行时间	运行时间方差	公平性评价
DEFAULT	110	1	优
RR	107	0	最优
PRIORITY	107	0	最优
FCFS	106	1	优
SML	107	1	优

这个场景专门验证 CPU 时间分配的公平性，结果符合预期。RR 和 PRIORITY 调度器的运行时间方差为 0，说明每个进程获得了完全相同的 CPU 时间，公平性达到最优。这对于 RR 来说是理所当然的——时间片轮转机制本身就是为公平性设计的。PRIORITY 调度器在所有进程优先级相同时退化为轮转行为，也表现出了很好的公平性。其他调度器的方差也只有 1，说明在进程完全相同的情况下，大多数调度算法都能做到较为公平的分配。

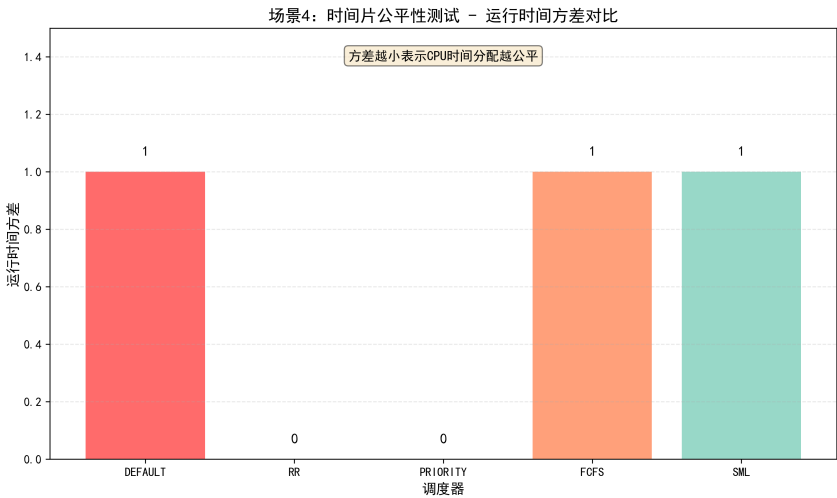


图 18 场景 4：时间片公平性测试 - 运行时间方差对比

该柱状图以方差作为公平性的量化指标，数值越低代表 CPU 时间分配越均匀。RR 和 PRIORITY 达到了 0 方差的理想状态，意味着所有进程获得完全相同的 CPU 时间。其他三个调度器的方差为 1，虽然略高但仍属于优秀范围。这个测试证明了在进程特征完全相同的情况下，现代调度算法都能较好地保证公平性。

综合性能对比

表 6 五种调度算法综合性能评估

评估维度	DEFAULT	RR	PRIORITY	FCFS	SML
短作业友好性	差	优	最优	中	优
交互响应性	差	中	最优	差	最优
低优先级公平性	差	优	中	优	差
高优先级响应	中	差	最优	中	最优
时间片公平性	优	最优	最优	优	优

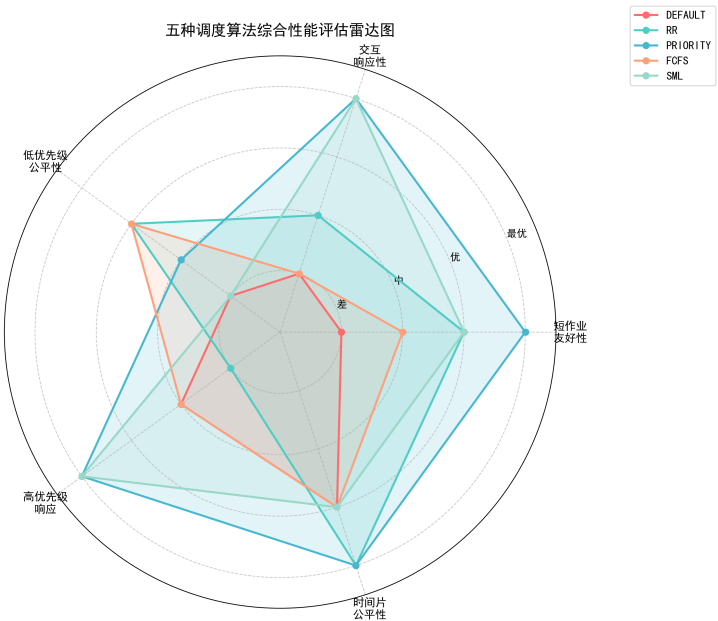


图 19 五种调度算法综合性能评估雷达图

雷达图以五个维度全面评估各调度器的性能特征。PRIORITY 调度器的覆盖面积最大，在短作业友好性、交互响应性和高优先级响应三个维度都达到最优，但在低优先级公平性上有所不足。RR 调度器的图形较为均衡，在时间片公平性上表现最佳，适合需要公平调度的场景。SML 综合了优先级和轮转的优点，在交互响应性和高优先级响应上达到最优。DEFAULT 调度器在所有维度上表现平庸。FCFS 在低优先级公平性上表现不错，但交互响应性较差。该图直观展示了”没有万能调度算法”的核心观点——应根据具体应用场景选择合适的调度策略。

5 实验中遇到的问题及解决方案

在本次实验过程中，遇到了多个技术问题。通过分析和调试，逐一找到了问题的根源并予以解决。以下是主要问题的记录与分析。

5.1 问题 1：运行 scheduler_test 时磁盘块耗尽

现象：运行 scheduler_test 测试程序时，内核 panic: "balloc: out of blocks"

原因分析：xv6 文件系统默认只有 1000 个块（约 500KB）。测试程序需要创建多个子进程，每个进程都会进行文件 IO 操作，加上结果输出文件的写入，很快就耗尽了磁盘空间。

解决方案：修改 param.h 中的文件系统大小常量：

```
1 define FSSIZE 10000 // 从 1000 增加到 10000
```

5.2 问题 2：PRIORITY/SML 调度器交互响应测试表现异常

现象：在场景 2 测试中，PRIORITY 调度器下高优先级的交互型任务（优先级 3）平均就绪时间反而高于低优先级的 CPU 背景任务（优先级 18），与预期相反。

这是由多个因素共同导致的：

1. **非抢占式调度：**PRIORITY 和 SML 调度器只在进程主动让出 CPU 时才重新调度。CPU 密集型任务没有 yield/sleep/exit 调用，会一直运行到完成。
2. **setpriority 时机问题：**测试程序中，子进程在 fork() 后才执行 setpriority()。在此之前，其他 CPU 密集型进程可能已经开始运行并占用 CPU。
3. **进程创建顺序：**测试按顺序先创建 CPU 背景任务，再创建交互型任务。当交互型任务被创建时，CPU 背景任务已在运行且不会主动让出。

解决方案：实现抢占式优先级调度，在时钟中断时检查是否有更高优先级进程就绪。

6 实验结论

本实验通过在 xv6 内核中实现并对比轮转（RR）、优先级（Priority）、先来先服务（FCFS）和静态多级队列（SML）四种调度算法，结合 test_scheduler.c 构造的四个典型测试场景，深入分析了不同调度策略对系统性能的影响。

实验数据表明，没有一种单一的简单调度算法能在所有场景下胜出。各算法的特性及适用场景总结如下：

表 7 调度算法特性与适用场景分析

调度算法	优势	劣势	适用场景
FCFS	实现简单,无进程切换开销	严重的车队效应,短作业等待时间长,交互体验差	批处理系统, 后台计算任务
RR (轮转)	响应时间短,公平性好(方差小), 防止饥饿	频繁切换开销大,未区分进程重要性,关键任务得不到优先处理	分时系统, 多用户交互环境
Priority	关键任务响应极快,支持不同服务等级	低优先级进程可能”饥饿”, 优先级反转风险	实时系统, 区分用户等级的系统
SML (多级队列)	结合了响应性和灵活性, 兼顾多种需求	静态队列配置死板,低优先级队列仍可能饥饿	复杂的通用操作系统

6.1 从实验走向现代调度：完全公平调度 (CFS)

通过实验我们发现，传统的调度算法往往陷入”公平性”与”交互性”的博弈中：

- RR 追求绝对的时间片公平，却牺牲了重要进程的优先权；
- Priority 追求重要性优先，却带来了严重的饥饿和不公平问题；
- SML 虽然试图折中，但静态的队列划分依然不够灵活，难以应对瞬息万变的负载。

这正是现代 Linux 内核抛弃这些传统算法的原因。为了同时解决上述问题，Linux 2.6.23 引入了完全公平调度器（Completely Fair Scheduler, CFS）。

CFS 不再使用离散的时间片和静态优先级，而是基于**虚拟运行时间** (vruntime) 的概念。它通过红黑树结构动态维护进程队列，每次总是选择 vruntime 最小的进程运行。这种设计带来了质的飞跃：

1. **动态公平：**不需要复杂的时间片计算，vruntime 自动体现了进程占用 CPU 的”不公平程度”，调度器只需不断补偿”最不公平”的进程。
2. **平滑优先级：**优先级被转化为权重，影响 vruntime 的增长速度。高优先级进程 vruntime 涨得慢，从而获得更多 CPU 时间，但不会像传统 Priority 那样完全饿死低优先级进程。
3. **交互优化：**睡眠的交互进程唤醒时 vruntime 较小，能立即抢占 CPU，天然保证了低延迟。

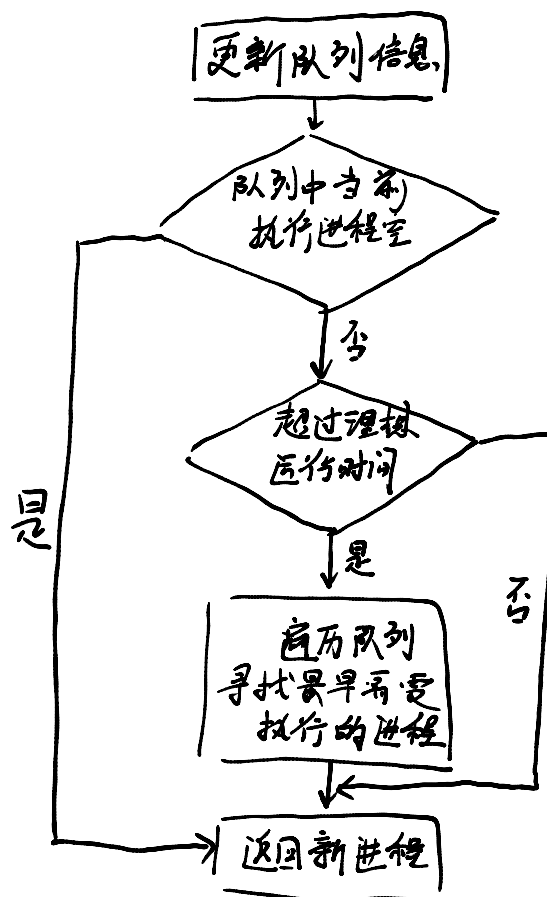


图 20 完全公平调度算法大概流程图

本次实验实现的算法是操作系统调度的基石，而理解它们的缺陷，正是理解 CFS 等现代复杂调度算法设计哲学的关键一把钥匙。

7 实验心得

这次实验让我对操作系统调度有了更深刻的认识。

我深刻体会到了调度算法的选择对系统行为的深远影响。在实现和测试不同调度算法的过程中，我发现看似简单的调度决策实际上涉及到响应时间、吞吐量、公平性等多个相互制约的因素。例如，优先级调度虽然能让高优先级进程快速响应，但如果不加限制，低优先级进程可能永远得不到执行机会，这就是所谓的饥饿问题。

此外，设计可扩展的调度框架让我对软件工程中的抽象和模块化有了新的理解。通过使用函数指针数组，我实现了调度算法的即插即用，这种设计使得添加新算法变得非常简单。这个经验让我意识到，好的架构设计能够极大地降低后续开发和维护的成本。

7.1 附录 A：调度器框架与数据结构

7.1.1 A.1 就绪队列数据结构

```
1 // 通用队列结构单元
2 struct procqueue {
3     struct proc *head;
4     struct proc *tail;
5 };
6
7 // FCFS/RR 队列 (1个)
8 struct procqueue fcfsQueue;
9
10 // Priority 最小堆, 优先级越小越靠前
11 struct priorityHeap {
12     struct proc *procs[NPROC]; // 堆数组
13     int size; // 当前堆大小
14 } pHeap;
15
16 // SML 队列 (3个: 0=高优先级 1-7, 1=中优先级 8-14, 2=低优先级 15-20)
17 struct procqueue smlQueues[3];
```

7.1.2 A.2 队列操作函数

```
1 // 入队: 将进程添加到队列尾部
2 void enqueue(struct procqueue *q, struct proc *p) {
3     p->next = 0;
4     if (q->tail == 0) {
5         q->head = q->tail = p;
6     } else {
7         q->tail->next = p;
8         q->tail = p;
9     }
10 }
11
12 // 出队: 从队列头部取出进程
13 struct proc *dequeue(struct procqueue *q) {
14     struct proc *p = q->head;
```

```
15     if (p == 0) return 0;
16     q->head = p->next;
17     if (q->head == 0) {
18         q->tail = 0;
19     }
20     p->next = 0;
21     return p;
22 }
```

7.1.3 A.3 统一入队函数

```
1 // 统一入队函数：进程变为 RUNNABLE 时调用
2 // 根据当前激活的调度器，只添加到对应的队列
3 extern int schedSelected; // 来自 sdh.h，表示当前调度器类型
4
5 void addToReadyQueues(struct proc *p) {
6     switch (schedSelected) {
7         case 0: // DEFAULT
8         case 2: // FCFS
9         case 3: // RR
10             enqueue(&fcfsQueue, p);
11             break;
12         case 1: // PRIORITY - 使用最小堆
13             heapInsert(p);
14             break;
15         case 4: // SML
16             if (p->priority >= 1 && p->priority <= 7) {
17                 enqueue(&smlQueues[0], p);
18             } else if (p->priority >= 8 && p->priority <= 14) {
19                 enqueue(&smlQueues[1], p);
20             } else {
21                 enqueue(&smlQueues[2], p);
22             }
23             break;
24         default:
25             enqueue(&fcfsQueue, p);
26             break;
27     }
```

```
28 }
```

7.2 附录 B：调度算法实现

7.2.1 B.1 优先级调度器 (Priority Scheduler)

```
1 // 优先级调度器：遍历进程表，选择优先级最高（数值最小）的就绪进程
2 // 时间复杂度  $O(n)$ ，但能实时反映优先级变化
3 struct proc *priorityScheduler() {
4     struct proc *p, *best = 0;
5     for (p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
6         if (p->state == RUNNABLE) {
7             if (best == 0 || p->priority < best->priority)
8                 best = p;
9         }
10    }
11    if (best != 0) best->state = RUNNING;
12    return best;
13 }
```

7.2.2 B.2 先来先服务调度器 (FCFS)

```
1 // 先来先服务调度器：从队列头部取出进程
2 // 非抢占式，进程运行直到主动让出 CPU
3 struct proc *fcfsScheduler() {
4     return dequeue(&fcfsQueue);
5 }
```

7.2.3 B.3 轮转调度器 (Round Robin)

```
1 // 轮转调度器：从FCFS队列取出，调度后由yield重新入队尾
2 // 与FCFS共用队列，但在trap.c中会被时钟中断抢占
3 struct proc *rrScheduler() {
4     return dequeue(&fcfsQueue);
5 }
```

7.2.4 B.4 静态多级队列调度器 (SML)

```
1 // 静态多级队列调度器：依次从高、中、低优先级队列出队
2 // 高优先级队列优先，同级别内轮转 (FCFS)
3 // 时间复杂度  $O(1)$ 
4 struct proc *smlScheduler() {
5     struct proc *p;
6
7     // 优先从高优先级队列出队
8     p = dequeue(&smlQueues[0]);
9     if (p != 0) return p;
10
11    // 高优先级队列空，尝试中优先级队列
12    p = dequeue(&smlQueues[1]);
13    if (p != 0) return p;
14
15    // 中优先级队列也空，尝试低优先级队列
16    return dequeue(&smlQueues[2]);
17 }
```

7.3 附录 C：调度器框架

7.3.1 C.1 调度器选择机制 (sdh.c)

```
1  #include "types.h"
2  #include "defs.h"
3  #include "proc.h"
4  #include "sdh.h"
5
6  // 全局调度器ID (默认为0)
7  int schedSelected = 0;
8
9  // 调度算法函数指针数组
10 struct proc *(*schedulerFunction[5])(void);
11
12 // 当前激活的调度函数指针
13 struct proc *(*ready_process)(void);
14
15 // 初始化调度器
16 void initScheduler(void) {
17     schedulerFunction[0] = defaultScheduler; // DEFAULT
18     schedulerFunction[1] = priorityScheduler; // PRIORITY
19     schedulerFunction[2] = fcfsScheduler;    // FCFS
20     schedulerFunction[3] = rrScheduler;      // RR
21     schedulerFunction[4] = smlScheduler;     // SML
22
23     ready_process = schedulerFunction[0];
24 }
25
26 // 切换调度器
27 int setscheduler(int schedId) {
28     if (schedId < 0 || schedId > 4) return -1;
29     schedSelected = schedId;
30     ready_process = schedulerFunction[schedId];
31     return 0;
32 }
```

7.3.2 C.2 调度器主循环 (scheduler 函数)

```
1 // scheduler - 调度器主循环函数
2 // 功能：操作系统的核心调度循环，负责选择并切换到下一个要执行的进程
3 // 说明：此函数永不返回，每个CPU核心启动后会一直在此循环中运行
4 void scheduler(void) {
5     struct proc *p;
6     struct cpu *c = mycpu();
7     c->proc = 0;
8
9     for (;;) {
10         // 步骤1：启用中断，防止死锁
11         sti();
12
13         // 步骤2：获取进程表锁
14         acquire(&ptable.lock);
15
16         // 步骤3：调用当前选定的调度算法选择下一个进程
17         acquire(&schedulerlock);
18         p = (*ready_process)();
19         release(&schedulerlock);
20
21         // 步骤4：如果找到了就绪进程，则进行上下文切换
22         if (p != 0) {
23             c->proc = p;
24             switchvm(p);           // 切换到进程的页表
25             p->state = RUNNING;
26             swtch(&(c->scheduler), p->context); // 上下文切换
27             switchkvm();           // 恢复内核页表
28             c->proc = 0;
29         }
30
31         // 步骤5：释放进程表锁
32         release(&ptable.lock);
33     }
34 }
```


7.3.3 C.3 进程让出 CPU (yield 函数)

```
1 // yield - 进程主动放弃 CPU 资源
2 // 用于实现轮转调度的时间片机制
3 void yield(void) {
4     acquire(&ptable.lock);
5     struct proc *curp = myproc();
6     curp->state = RUNNABLE;
7     addToReadyQueues(curp); // 重新加入就绪队列
8     sched();                // 切换到调度器
9     release(&ptable.lock);
10 }
```

7.4 附录 D：时钟中断与进程统计

7.4.1 D.1 时钟中断处理 (trap.c)

```
1 void trap(struct trapframe *tf) {
2     // ... 省略系统调用处理 ...
3
4     switch (tf->trapno) {
5     case T_IRQ0 + IRQ_TIMER: // 时钟中断
6         if (cpuid() == 0) {
7             acquire(&tickslock);
8             ticks++; // 增加全局时钟计数
9             update_statistics(); // 更新进程时间统计
10            wakeup(&ticks); // 唤醒等待时钟的进程
11            release(&tickslock);
12        }
13        lapiceoi();
14        break;
15     // ... 省略其他中断处理 ...
16     }
17
18     // 抢占式调度：时间片用完就让出 CPU
19     if (myproc() && myproc()->state == RUNNING &&
20         tf->trapno == T_IRQ0 + IRQ_TIMER) {
```

```
21     yield(); // 无条件让出CPU, 由调度器决定下一个进程
22 }
23 }
```

7.4.2 D.2 进程时间统计更新

```
1 // update_statistics - 更新所有进程的时间统计
2 // 每个时钟中断调用一次
3 void update_statistics(void) {
4     struct proc *p;
5     acquire(&ptable.lock);
6
7     for (p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
8         switch (p->state) {
9             case RUNNING:
10                 p->rtime++; // 运行时间 +1
11                 break;
12             case RUNNABLE:
13                 p->retime++; // 就绪等待时间 +1
14                 break;
15             case SLEEPING:
16                 p->stime++; // 睡眠时间 +1
17                 break;
18             default:
19                 break;
20         }
21     }
22
23     release(&ptable.lock);
24 }
```

7.4.3 D.3 wait2 系统调用

```
1 // wait2 - wait的增强版, 返回子进程的时间统计信息
2 int wait2(int *retime, int *runtime, int *stime) {
3     struct proc *p;
4     int havekids, pid;
5
6     acquire(&ptable.lock);
7     for (;;) {
8         havekids = 0;
9         for (p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
10             if (p->parent != myproc()) continue;
11             havekids = 1;
12
13             if (p->state == ZOMBIE) {
14                 // 收集时间统计信息
15                 *retime = p->retime; // 就绪等待时间
16                 *runtime = p->runtime; // 实际运行时间
17                 *stime = p->stime; // 睡眠时间
18
19                 pid = p->pid;
20                 // 释放进程资源...
21                 kfree(p->kstack);
22                 p->kstack = 0;
23                 freevm(p->pgdir);
24                 p->state = UNUSED;
25                 // 清零统计字段...
26
27                 release(&ptable.lock);
28                 return pid;
29             }
30         }
31
32         if (!havekids || myproc()->killed) {
33             release(&ptable.lock);
34             return -1;
35         }
36
37         sleep(myproc(), &ptable.lock);
38     }
39 }
```

8 附录：完整实验结果

8.1 场景 1：纯 CPU 长短作业（车队效应测试）

表 8 场景 1 - DEFAULT 调度器测试结果

PID	优先级	类型	就绪时间	运行时间	周转时间
4	10	长作业	24	219	243
5	10	长作业	263	218	481
6	10	短作业	460	22	550
7	10	短作业	471	22	623
8	10	中作业	478	75	621
9	10	中作业	538	75	627
长作业平均			143	218	362
短作业平均			465	22	487
中作业平均			508	75	583

表 9 场景 1 - RR(轮转) 调度器测试结果

PID	优先级	类型	就绪时间	运行时间	周转时间
30	10	短作业	102	21	123
31	10	短作业	147	22	382
32	10	中作业	266	74	370
33	10	中作业	266	72	370
28	10	长作业	407	225	639
29	10	长作业	408	216	637
长作业平均			407	220	628
短作业平均			124	21	146
中作业平均			266	73	339

表 10 场景 1 - PRIORITY 调度器测试结果

PID	优先级	类型	就绪时间	运行时间	周转时间
52	10	长作业	215	217	432
53	10	长作业	215	214	429
54	10	短作业	24	21	45
55	10	短作业	21	22	43
56	10	中作业	73	71	144
57	10	中作业	73	74	147
长作业平均			215	215	430
短作业平均			22	21	44
中作业平均			73	72	145

表 11 场景 1 - FCFS 调度器测试结果

PID	优先级	类型	就绪时间	运行时间	周转时间
76	10	长作业	1	214	215
77	10	长作业	214	214	428
78	10	短作业	0	22	22
79	10	短作业	22	22	44
80	10	中作业	1	71	72
81	10	中作业	72	72	144
长作业平均			107	214	321
短作业平均			11	22	33
中作业平均			36	71	108

表 12 场景 1 - SML(多级队列) 调度器测试结果

PID	优先级	类型	就绪时间	运行时间	周转时间
102	10	短作业	106	21	134
103	10	短作业	110	22	134
104	10	中作业	287	72	515
100	10	长作业	404	227	631
101	10	长作业	407	216	631
105	10	中作业	286	71	515
长作业平均			405	221	627
短作业平均			108	21	129
中作业平均			286	71	358

8.2 场景 2：交互型 vs 背景任务

表 13 场景 2 - DEFAULT 调度器测试结果

PID	优先级	类型	就绪时间	运行时间	休眠时间	周转时间
10	18	CPU 背景	9	217	225	451
11	18	CPU 背景	228	215	9	452
12	3	交互型	440	80	189	709
13	3	交互型	514	81	121	716
14	12	IO 背景	593	45	222	860
15	12	IO 背景	265	46	556	867
交互型平均			477	80	155	712
CPU 背景平均			118	216	0	334
IO 背景平均			429	45	389	863

表 14 场景 2 - RR(轮转) 调度器测试结果

PID	优先级	类型	就绪时间	运行时间	休眠时间	周转时间
36	3	交互型	398	81	86	565
37	3	交互型	398	81	86	565
34	18	CPU 背景	447	236	6	689
35	18	CPU 背景	451	215	23	689
38	12	IO 背景	458	44	275	777
39	12	IO 背景	455	45	301	801
交互型平均			398	81	86	565
CPU 背景平均			449	225	0	674
IO 背景平均			456	44	288	789

表 15 场景 2 - PRIORITY 调度器测试结果

PID	优先级	类型	就绪时间	运行时间	休眠时间	周转时间
60	3	交互型	73	80	200	353
61	3	交互型	72	81	200	353
62	12	IO 背景	230	41	396	667
63	12	IO 背景	225	40	448	713
58	18	CPU 背景	475	236	10	721
59	18	CPU 背景	480	232	13	725
交互型平均			72	80	200	353
CPU 背景平均			477	234	0	711
IO 背景平均			227	40	422	690

表 16 场景 2 - FCFS 调度器测试结果

PID	优先级	类型	就绪时间	运行时间	休眠时间	周转时间
82	18	CPU 背景	1	215	0	216
83	18	CPU 背景	216	214	0	430
84	3	交互型	523	80	108	711
85	3	交互型	538	84	96	718
86	12	IO 背景	592	41	222	855
87	12	IO 背景	597	41	226	864
交互型平均			530	82	102	714
CPU 背景平均			108	214	0	323
IO 背景平均			594	41	224	859

表 17 场景 2 - SML(多级队列) 调度器测试结果

PID	优先级	类型	就绪时间	运行时间	休眠时间	周转时间
108	3	交互型	73	80	200	353
109	3	交互型	77	80	196	353
110	12	IO 背景	240	40	367	647
106	18	CPU 背景	477	239	0	716
107	18	CPU 背景	480	233	9	722
111	12	IO 背景	223	41	467	731
交互型平均			75	80	198	353
CPU 背景平均			478	236	0	714
IO 背景平均			231	40	417	689

8.3 场景 3：优先级饥饿测试

表 18 场景 3 - DEFAULT 调度器测试结果

PID	优先级	优先级组	就绪时间	运行时间	周转时间
16	1	高优先级	40	436	476
17	10	中优先级	481	111	592
18	10	中优先级	586	110	696
19	10	中优先级	688	110	798
20	19	低优先级	780	111	891
21	19	低优先级	880	112	992
高优平均			40	436	476
中优平均			585	110	695
低优平均			830	111	941

表 19 场景 3 - RR(轮转) 调度器测试结果

PID	优先级	优先级组	就绪时间	运行时间	周转时间
41	10	中优先级	539	108	647
42	10	中优先级	537	107	644
43	10	中优先级	537	107	644
44	19	低优先级	540	108	648
45	19	低优先级	584	108	933
40	1	高优先级	540	446	988
高优平均			540	446	986
中优平均			537	107	645
低优平均			562	108	670

表 20 场景 3 - PRIORITY 调度器测试结果

PID	优先级	优先级组	就绪时间	运行时间	周转时间
64	1	高优先级	1	427	428
66	10	中优先级	124	107	301
65	10	中优先级	554	107	805
67	10	中优先级	217	113	377
69	19	低优先级	409	112	521
68	19	低优先级	442	113	556
高优平均			1	427	428
中优平均			298	109	407
低优平均			425	112	538

表 21 场景 3 - FCFS 调度器测试结果

PID	优先级	优先级组	就绪时间	运行时间	周转时间
88	1	高优先级	2	437	439
89	10	中优先级	439	108	547
90	10	中优先级	547	107	654
91	10	中优先级	654	109	763
92	19	低优先级	762	107	869
93	19	低优先级	869	108	977
高优平均			2	437	439
中优平均			546	108	654
低优平均			815	107	923

表 22 场景 3 - SML(多级队列) 调度器测试结果

PID	优先级	优先级组	就绪时间	运行时间	周转时间
112	1	高优先级	1	429	430
114	10	中优先级	648	108	756
115	10	中优先级	648	107	755
113	10	中优先级	647	113	899
116	19	低优先级	870	112	982
117	19	低优先级	871	114	989
高优平均			1	429	430
中优平均			647	109	757
低优平均			870	113	983

8.4 场景 4：时间片公平性测试

表 23 场景 4 - DEFAULT 调度器测试结果

PID	优先级	就绪时间	运行时间	周转时间
22	10	10	109	119
23	10	132	110	242
24	10	250	109	359
25	10	355	110	465
26	10	455	112	567
27	10	550	111	661
平均运行时间			110	
运行时间方差			1	

表 24 场景 4 - RR(轮转) 调度器测试结果

PID	优先级	就绪时间	运行时间	周转时间
46	10	537	108	645
47	10	534	107	641
48	10	537	108	646
49	10	537	107	646
50	10	537	107	646
51	10	537	108	646
平均运行时间			107	
运行时间方差			0	

表 25 场景 4 - PRIORITY 调度器测试结果

PID	优先级	就绪时间	运行时间	周转时间
70	10	109	108	649
71	10	539	108	652
72	10	539	107	653
73	10	326	107	657
75	10	109	107	652
74	10	326	108	667
平均运行时间			107	
运行时间方差			0	

表 26 场景 4 - FCFS 调度器测试结果

PID	优先级	就绪时间	运行时间	周转时间
94	10	2	107	109
95	10	109	106	215
96	10	214	108	322
97	10	322	107	429
98	10	429	106	535
99	10	535	107	642
平均运行时间			106	
运行时间方差			1	

表 27 场景 4 - SML(多级队列) 调度器测试结果

PID	优先级	就绪时间	运行时间	周转时间
119	10	539	108	647
120	10	538	107	645
118	10	540	109	653
121	10	538	107	645
122	10	540	109	653
123	10	539	107	646
平均运行时间			107	
运行时间方差			1	

8.5 各场景调度算法性能汇总对比

表 28 场景 1：各调度算法平均性能对比（单位：ticks）

调度器	长作业			短作业			中作业		
	就绪	运行	周转	就绪	运行	周转	就绪	运行	周转
DEFAULT	143	218	362	465	22	487	508	75	583
RR	407	220	628	124	21	146	266	73	339
PRIORITY	215	215	430	22	21	44	73	72	145
FCFS	107	214	321	11	22	33	36	71	108
SML	405	221	627	108	21	129	286	71	358

表 29 场景 2：各调度算法平均性能对比（单位：ticks）

调度器	交互型			CPU 背景			IO 背景		
	就绪	休眠	周转	就绪	运行	周转	就绪	休眠	周转
DEFAULT	477	155	712	118	216	334	429	389	863
RR	398	86	565	449	225	674	456	288	789
PRIORITY	72	200	353	477	234	711	227	422	690
FCFS	530	102	714	108	214	323	594	224	859
SML	75	198	353	478	236	714	231	417	689

表 30 场景 3：各调度算法平均性能对比（单位：ticks）

调度器	高优先级		中优先级		低优先级	
	就绪	周转	就绪	周转	就绪	周转
DEFAULT	40	476	585	695	830	941
RR	540	986	537	645	562	670
PRIORITY	1	428	298	407	425	538
FCFS	2	439	546	654	815	923
SML	1	430	647	757	870	983

表 31 场景 4：各调度算法时间片公平性对比

调度器	平均运行时间	运行时间方差	公平性评价
DEFAULT	110	1	优
RR	107	0	最优
PRIORITY	107	0	最优
FCFS	106	1	优
SML	107	1	优